Multi-Agent Banking Architecture & Integration Document

End-to-End Query Lifecycle, Function Call Hierarchy, Data Flow & Step Validation Engine

Author: Lead System Architect | Primary LLM: Gemini 3.7 Flash | Fallback: Ollama phi3:mini | Curren

1. EXECUTIVE ARCHITECTURE OVERVIEW

The Multi-Agent Banking Harness is an enterprise-grade agentic pipeline executing under a strict hierarchy governed by Boss EVA (Cabin 0x1) and dedicated sub-agents: Specialist VK (Desk 1: Balance Inquiry) and Specialist RO (Desk 2: Account Statements). All database operations query PostgreSQL in Indian Rupees (INR / ¹) with zero database access for gree fallback: Google Gemini 3.7 Flash as primary, failing over to local Ollama (phi3:mini). Every step is audited with raw request/response logging and written to persistent logs for real-time validation.

2. FUNCTION CALL SEQUENCE & DATA TRANSFORMATION FLOW

STEP 1 — Customer Query Ingestion & Dynamic Account Parsing

fn: extract_account_id_from_text(prompt)

API: Client UI -> POST /api/orchestrator/dispatch (fastapi_app.py)

Action: Extracts ACC-XXXXX pattern via NLP regex. If missing, null is returned.

```
Output: { prompt: string, accountId: string | null }
```

STEP 2 — Dynamic Intent Classification (2-Tier LLM Pipeline)

fn: plan_orchestration() -> invoke_llm_with_fallback()

API: Internal LLM Call (Gemini 3.7 Flash -> Ollama phi3:mini fallback)

Action: Classifies intent: balance_inquiry, account_statement, greetings, or other. Zero database access during classification.

```
Output: { intent: string, confidence: number, reasoning: string, assignedAgent: string, subtasks: Array }
```

STEP 3A — Greetings & Conversational Flow (Zero DB Access Rule)

fn: general_or_greeting_node() -> synthesize_customer_response()

API: FastAPI Internal Route -> Direct Cabin 0x1 Delivery

Action: Boss EVA delivers direct welcoming message. Zero sub-agents dispatched and zero DB access.

```
Output: { success: true, customer_response: string, database_accessed: false }
```

STEP 3B — Specialist VK Subtask Execution (Balance Inquiry)

fn: agent_vk_node() -> execute_agent_subtask() -> get_account_data()

API: POST /api/agent/execute_subtask (Subtask ID: subtask_vk_1)

Action: Agent VK queries PostgreSQL accounts table for checking/savings balances, reconciles pending holds

```
Output: { speechSummary: string, code: { filename: "agent_vk_balance.py", ... }, thoughtLog: Array }
```

STEP 3C — Specialist RO Subtask Execution (Account Statement)

fn: agent_ro_node() -> execute_agent_subtask() -> get_transactions_data()

API: POST /api/agent/execute_subtask (Subtask ID: subtask_ro_1)

Action: Agent RO queries PostgreSQL transactions table, calculates total credits, total debits, net cashflow i

```
Output: { speechSummary: string, code: { filename: "agent_ro_statement.py", ... }, thoughtLog: Array }
```

STEP 4 **Boss EVA Final Customer Response Synthesis (INR Standard)**

`fn: boss_eva_synthesize_node() -> synthesize_customer_response()`

API: POST /api/eva/synthesize (fastapi_app.py)

Action: Formats verified ledger output into polite banking summary with INR currency symbols (₹ / INR). Enfo

Output: { success: true, customer_response: string, currencyValidated: true }

STEP 5 **Step Logger & Comprehensive Audit Validation**

`fn: write_to_file_log() & log_audit_to_postgres()`

API: Internal Logger -> logs/banking_audit.log & audit_logs table

Action: Appends structured validation entry to server logs and database audit table for immediate troubleshooting.

Output: Validated audit log entry appended to disk & PostgreSQL

3. CORE ARCHITECTURAL INVARIANTS & INTEGRATION POLICIES

• Zero Database Access for Greetings: When a user query is classified as a greeting, the system delivers direct responses without database invocation.

• Strict 2-Tier Fallback Hierarchy: LLM requests attempt Google Gemini 3.7 Flash first. If absent or failing, requests fall back seamlessly to local Ollama (phi3:mini).

• Strict INR Currency Standard: All monetary transactions, account balances, inflows, and outflows are computed and displayed exclusively in Indian Rupees (₹ / INR).

• Dynamic Subtask Planning: The orchestrator dynamically assigns subtasks based on natural language intent (Balance -> VK, Statement -> RO).

• Complete Auditability: Raw LLM request payloads, responses, and execution steps are captured at every invocation for full traceability.